SESSION 6 – AADK TASK 6

Deep Dive into UI State (Jetpack Compose)

Name: Sachin
USN: 1JS22CS133
Semester: 8th Semester
Course: Android App Development using Gen AI
Date: 26 February 2026

---

1. Aim

To understand and implement structured UI state management in Jetpack Compose by building a dynamic screen that responds to user interaction, state changes, and configuration changes.

---

2. Feature Description

This application demonstrates state-driven UI using Jetpack Compose.
The screen contains two dice values, a roll counter, a Roll button, and a Reset button. When the user clicks the Roll button, the dice values change randomly and the UI updates automatically. If both dice show the same number, a special message is displayed. The Reset button restores the initial state.

This task demonstrates how user interaction triggers state updates and recomposition in Compose.

---

3. UI Layout Overview

The screen contains:

- Title Text ("Dice Roller")
- Dice 1 value display
- Dice 2 value display
- Roll count display
- Roll Dice button
- Reset button
- Message display for double condition

The layout is structured using:

- Column
- Text
- Button
- Spacer
- Modifier

---

4. State Variables Used

The following state variables are used to control the UI:

- `dice1` → Stores first dice value
- `dice2` → Stores second dice value
- `rollCount` → Tracks number of times the button is pressed
- `message` → Displays special feedback when both dice match

State is defined using:

```
var dice1 by remember { mutableStateOf(1) } var
dice2 by remember { mutableStateOf(1) } var
rollCount by remember { mutableStateOf(0) } var
message by remember { mutableStateOf("") }
```

These variables control the UI dynamically.

---

5. Complete Compose Implementation

```
@Composable
fun DiceRollerScreen() {
     var dice1 by remember { mutableStateOf(1) }
var dice2 by remember { mutableStateOf(1) }     var
rollCount by remember { mutableStateOf(0) }     var
message by remember { mutableStateOf("") }
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
verticalArrangement = Arrangement.Center     ) {

        Text(text = "Dice Roller", style =
MaterialTheme.typography.headlineMedium)

        Spacer(modifier = Modifier.height(20.dp))
        Text(text = "Dice 1: $dice1")
        Text(text = "Dice 2: $dice2")
        Text(text = "Roll Count: $rollCount")
        Spacer(modifier = Modifier.height(20.dp))
```

```
        Button(onClick = {
dice1 = (1..6).random()
dice2 = (1..6).random()
rollCount++

            message = if (dice1 == dice2) {
                "Double!"
            } else {
                ""
            }
        }) {
            Text("Roll Dice")
        }

        Spacer(modifier = Modifier.height(10.dp))

        Button(onClick = {
dice1 = 1             dice2
= 1
            rollCount = 0
message = ""
        }) {
            Text("Reset")
        }

        Spacer(modifier = Modifier.height(20.dp))

        Text(text = message)
    }
}
```

---

6. How State Controls UI (Recomposition Flow)

When the Roll button is clicked:

1. The onClick handler executes.
2. State variables (`dice1`, `dice2`, `rollCount`, `message`) are updated.
3. Compose detects state change.
4. Only the affected UI elements are recomposed.
5. Updated values appear instantly on screen.

This demonstrates the flow:

User Interaction → State Update → Recomposition → UI Update

---

7. Handling Configuration Changes

To preserve state during screen rotation or configuration changes, `rememberSaveable` can be used instead of `remember`.

Example: `var dice1 by rememberSaveable {`

```
mutableStateOf(1) }
```

This ensures that values are retained even when the device rotates.

---

8. State Model Summary

State Variable	Purpose
dice1	Stores first dice value
dice2	Stores second dice value
rollCount	Tracks total rolls
message	Shows feedback if double occurs

---

9. Reflection

This task helped me understand how Jetpack Compose is fully state-driven. Any change in state automatically updates the UI without manual refresh. Proper state management is important to avoid inconsistent UI behaviour. I learned how recomposition works and how structured state improves maintainability and readability of code. This exercise increased my confidence in building dynamic Compose interfaces.

---

10. Conclusion

The Dice Roller application successfully demonstrates structured state management in Jetpack Compose. The UI dynamically adapts to user interaction, and state changes trigger automatic recomposition. This task strengthened understanding of interactive UI design and state-driven development.